

Note di lavoro: cosa rende un grafo “difficile” per imparare la connettività

9 giugno 2026

Questo documento spiega, passo per passo e in parole semplici, il ragionamento dietro i prossimi esperimenti. Niente gergo non spiegato.

1 Due domande diverse che stiamo inseguendo

Stiamo facendo due cose distinte. È importante non confonderle.

Domanda A — “possiamo fare un modello migliore?” Lo standard transformer (quello del paper) generalizza male. La domanda A è: *se cambiamo un pezzo dell’architettura, otteniamo un modello che sbaglia meno?* Il pezzo che stiamo cambiando è il **read-out** (lo spiego tra poco). Questa domanda è “costruttiva”: vogliamo un modello migliore.

Domanda B — “cosa rende un grafo difficile?” Il paper sostiene che la difficoltà di un grafo sia il suo **diametro** (la distanza massima tra due nodi). La domanda B è: *è davvero il diametro? O c’è una proprietà del grafo che predice meglio quando il modello sbaglia?* Questa domanda è “descrittiva”: vogliamo capire, non costruire.

Come si parlano. Sono separate, ma collegate: se con la domanda B capiamo *perché* certi grafi sono difficili, allora sappiamo *quale* modifica provare nella domanda A. Esempio: se scopriamo che la difficoltà sono i “colli di bottiglia” del grafo, allora un’architettura che aiuta a superarli è la cosa giusta da costruire.

2 Come funziona il modello, in parole semplici

- **Input:** la matrice di adiacenza (chi è collegato a chi).
- Il transformer costruisce, per ogni nodo, un **embedding**: un vettore di numeri che “riassume” quel nodo.
- Il **read-out** è la regola che, guardando gli embedding di due nodi h_i e h_j , decide: “sono nella stessa componente, sì o no?”

Usiamo due read-out diversi:

- **Lineare** (quello standard): una formula generica che combina i due vettori. Non richiede che i due vettori si somiglino.

- **Similarità:** dice “connessi” se i due vettori **puntano nella stessa direzione** (coseno alto tra h_i e h_j).

3 L’idea chiave: con la similarità, “connesso = stesso vettore”

Se uso il read-out a **similarità**, il modello ideale deve fare questo: dare a tutti i nodi della *stessa* componente un embedding (quasi) identico, e a nodi di componenti *diverse* embedding diversi.

Analogia. Pensa a colorare i nodi. Ogni componente connessa è un colore. Il modello ideale dà lo stesso colore a tutti i nodi collegati tra loro. Allora “ i e j connessi?” diventa semplicemente “hanno lo stesso colore?”.

Attenzione: questo vale *solo* con il read-out a similarità. Con il read-out lineare il modello *non* ha bisogno di rendere uguali gli embedding, quindi guardare “quanto sono simili i vettori” lì non ci dice nulla di utile. Questo è un punto importante e ci torneremo.

4 Perché alcuni grafi sono difficili: l’informazione deve “viaggiare”

Come fa il modello a dare lo stesso embedding (colore) a due nodi connessi ma *lontani*? L’informazione deve propagarsi lungo il grafo, di vicino in vicino. **Ogni layer del transformer è un passo di propagazione:** come una voce che si passa di persona in persona.

Il nostro modello ha 2 layer, quindi fa circa 2 passi. Allora:

- Se due nodi connessi sono “facili da raggiungere” (tanti cammini, ben collegati), 2 passi bastano a dargli lo stesso colore $\rightarrow$ facile.
- Se sono separati da un **collo di bottiglia** (un solo ponte stretto) o da un lungo corridoio, 2 passi *non* bastano $\rightarrow$ il modello non riesce a dargli lo stesso colore $\rightarrow$ sbaglia.

Nei modelli su grafi questo fenomeno ha un nome: **over-squashing** (l’informazione si “strozza” nei colli di bottiglia).

Esempi concreti (dai nostri dati reali).

grafo	com’è fatto	diametro	quanto sbaglia (exact)
expander	tutti ben collegati	5	0.99 (facile)
clique	tutti con tutti	1	facile (sul reach)
catena lunga (1chain)	fila indiana	19	0.45
barbell	due gruppi + ponte stretto	11	0.22 (difficile)

Guarda l’ultimo confronto: il **barbell** ha diametro *più piccolo* (11) della catena lunga (19), eppure è *più difficile* (0.22 contro 0.45)! Quindi il diametro *non* spiega la difficoltà: il vero problema del barbell è il ponte stretto, non la distanza. Questo è il cuore della domanda B.

5 Come misurare la difficoltà: tre candidati

(a) Diametro — la proposta del paper. La distanza massima (in numero di passi) tra due nodi. *Limite:* conta solo i passi, non *quanti* cammini ci sono. Due nodi a 5 passi lungo

un corridoio (un solo cammino) sono molto più difficili di due nodi a 5 passi dentro un gruppo denso (mille cammini). Il diametro non vede questa differenza.

(b) Spectral gap — “quanto in fretta si mescola il grafo”. Un singolo numero per tutto il grafo. Grande = l’informazione si mescola in fretta (facile); piccolo = c’è un collo di bottiglia (difficile). È utile, ma è un solo numero per l’intero grafo, non per la singola coppia di nodi.

(c) Resistenza effettiva — per ogni coppia di nodi. Immagina il grafo come una rete di fili elettrici, ogni arco è una resistenza. La “resistenza effettiva” tra due nodi misura quanto è difficile far passare corrente tra loro.

- Tanti cammini paralleli $\rightarrow$ bassa resistenza $\rightarrow$ facile.
- Un solo cammino lungo, o un collo di bottiglia $\rightarrow$ alta resistenza $\rightarrow$ difficile.

Analogia: due città collegate da 10 autostrade contro due città collegate da 1 strada sterrata. La “distanza” può essere la stessa, ma la difficoltà di spostarsi è diversa. La resistenza coglie questa differenza, il diametro no. Ed è una misura **per coppia di nodi**, quindi più fine del diametro.

(d) P^L — la più fedele al nostro modello. P vuol dire “fai un passo casuale verso un vicino”. P^L vuol dire “dopo L passi, quanta probabilità di essere arrivato dall’altro nodo”. Mettiamo $L = 2$ perché il modello ha 2 layer (= 2 passi). Se dopo 2 passi non ci si arriva quasi mai, allora un modello a 2 layer *non può* aver collegato quei due nodi $\rightarrow$ coppia difficile. Il bello di questa misura è che usa esattamente il “budget di passi” del modello (cioè il numero di layer).

6 L’esperimento che voglio fare

Prendo i modelli già allenati. Per ogni coppia di nodi *connessi*, raccolgo:

1. quanto il modello ha sbagliato su quella coppia;
2. (solo per il read-out a similarità) quanto sono diventati simili i loro embedding — cioè se il modello è riuscito a dargli lo stesso “colore”;
3. le tre misure di difficoltà: diametro, resistenza effettiva, P^L .

Poi guardo: **quale misura predice meglio l’errore?** Se la resistenza (o P^L) batte il diametro, abbiamo trovato la nozione giusta di difficoltà — e questo è esattamente il punto delle domande B (capire) e A (sapere cosa migliorare).

7 Le due scelte che ti chiedevo (spiegate bene)

Scelta 1: pesi “uniformi” o pesi “veri” del modello?

Quando il modello propaga l’informazione, *non* dà lo stesso peso a tutti i vicini: il meccanismo di attention decide a chi “dare retta” di più, e questi pesi li impara durante il training. Quando calcolo P^L (la propagazione) ho due strade:

- **Uniforme** (semplice): faccio finta che ogni nodo tratti tutti i vicini allo stesso modo. Dipende solo dal grafo, non dal modello. Facile e veloce da calcolare.
- **Pesi veri**: uso i veri pesi di attention che il modello ha imparato. Più fedele a ciò che il modello fa davvero, ma più lavoro.

La frase “proxy falsificabile” che non ti diceva niente significa questo: *proxy* = approssimazione semplice (la versione uniforme); *falsificabile* = fa una previsione netta (“le coppie con propagazione bassa sono quelle che il modello sbaglia”) che possiamo verificare e, se è sbagliata, scoprirlo subito.

Proposta: parto dalla versione *uniforme* perché è semplice. Se funziona, ci fermiamo lì. Se non basta a spiegare gli errori, allora passo ai *pesi veri* del modello. In breve: non complico finché non serve.

Scelta 2: come confronto la resistenza tra grafi diversi?

La resistenza ha valori su scale diverse a seconda di quanto è grande il grafo, quindi confrontare i numeri “grezzi” tra grafi diversi è ingannevole. Soluzione: invece dei valori assoluti guardo l’**ordine** (quale coppia è più difficile di quale) e confronto gli ordini. Così la scala non mi inganna.

8 Un’ultima distinzione: “reach” contro “cut”

Ci sono due compiti diversi dentro la connettività:

- **Reach**: rendere *uguali* due nodi connessi ma lontani. La difficoltà qui è la resistenza / il collo di bottiglia (tutto quanto detto sopra).
- **Cut**: rendere *diversi* due nodi di componenti separate. Strutturalmente è facile (sono scollegati). Quando il modello sbaglia qui (es. sulle due cricche), è per un altro motivo — una scorciatoia tipo “ha tanti vicini, quindi lo dico connesso” — non per un collo di bottiglia.

Per questo, nell’analisi, tengo separate le coppie “reach” dalle coppie “cut”: le misure di difficoltà di cui sopra spiegano il *reach*; il *cut* è un problema diverso che va guardato a parte.

In sintesi

1. Con il read-out a similarità, il modello ideale dà lo stesso “colore” (embedding) a nodi connessi. Sì, la tua intuizione era giusta.
2. Alcuni grafi sono difficili perché l’informazione fatica a viaggiare tra nodi connessi (colli di bottiglia) — è l’over-squashing.
3. Il diametro non basta a misurarlo (il barbell lo dimostra). Candidati migliori: spectral gap, resistenza effettiva, P^L .
4. L’esperimento: vedere quale misura predice gli errori del modello.
5. Le due scelte: parto dalla versione semplice (uniforme) e uso l’ordine invece dei valori grezzi.